Entrega 2 Bases de Datos 2

TurismoUQ

Juan Pablo Galeano Correa

Daniel Estiven Garcia

Función valor estadía

Se descarta el uso de bucles para calcular el precio noche por noche, ya que iterar fila por fila afecta el rendimiento de la base de datos. Se optó por una operación de conjuntos que calcula el total en un solo paso mediante álgebra de fechas.

La función determina cuántas noches de la estadía caen dentro de cada temporada cruzando los rangos. Usa GREATEST(fecha_in, fecha_inicio) para encontrar el límite inferior real del cobro en esa temporada, y LEAST(fecha_out, fecha_fin + 1) para el límite superior.

```
CREATE OR REPLACE FUNCTION fn_valor_estadia (
    p_id_habitacion IN NUMBER,
    p_fecha_in      IN DATE,
    p_fecha_out     IN DATE
) RETURN NUMBER IS
    v_total NUMBER := 0;
BEGIN

    IF p_fecha_out <= p_fecha_in THEN
        RETURN 0;
    END IF;


    SELECT NVL(SUM(
              (LEAST(p_fecha_out, t.fecha_fin + 1) - GREATEST(p_fecha_in, t.fecha_inicio)) * tf.valor_noche
          ), 0)
      INTO v_total
      FROM temporada t
      JOIN tarifa tf ON tf.id_temporada = t.id_temporada
     WHERE tf.id_habitacion = p_id_habitacion
       AND t.fecha_inicio < p_fecha_out
       AND t.fecha_fin >= p_fecha_in;

    RETURN v_total;
END fn_valor_estadia;
/
```

Prueba

- ID Habitación: 10.
- Check-in (fecha_in): 10 de diciembre de 2025.
- Check-out (fecha_out): 15 de diciembre de 2025.

La temporada alta "Diciembre-enero 2025-2026" inicia el 13 de diciembre de 2025. La función facturará las noches del 10, 11 y 12 con la tarifa vigente para esa fecha (media o baja), y las noches del 13 y 14 con el recargo de la temporada alta.

```sql
SELECT 10 AS id_habitacion,
       TO_DATE('2025-12-10', 'YYYY-MM-DD') AS fecha_checkin,
       TO_DATE('2025-12-15', 'YYYY-MM-DD') AS fecha_checkout,
       fn_valor_estadia(
         10,
         TO_DATE('2025-12-10', 'YYYY-MM-DD'),
         TO_DATE('2025-12-15', 'YYYY-MM-DD')
       ) AS valor_total_estadia
  FROM dual;
```

ID_HABITACION	FECHA_CHECKIN	FECHA_CHECKOUT	VALOR_TOTAL_ESTADIA
10	10/12/25	15/12/25	2339000

Procedimiento crear reserva

Orden de evaluación: Las reglas de negocio se validan de menor a mayor costo en la base de datos. Primero se compara en memoria la aritmética de fechas (-20001), luego se consulta la capacidad del registro maestro (-20002), y finalmente se realiza el cruce relacional de solapes sobre las tablas de movimiento (-20003).

Códigos de excepción personalizados: Se implementa RAISE_APPLICATION_ERROR dentro del rango estándar de Oracle (-20000 a -20999), garantizando que la aplicación cliente o el entorno de pruebas identifique con precisión el motivo del rechazo:

- -20001: Fechas invertidas o iguales.
- -20002: Capacidad máxima superada.
- -20003: Conflicto de ocupación de habitación.

Al crear la reserva, el procedimiento obtiene el id_alojamiento directamente de la habitación para poblar la cabecera RESERVA, e inserta concurrentemente en RESERVA_HABITACION usando la cláusula RETURNING INTO. Esto mantiene la integridad referencial y deja el estado inicial como 'PENDIENTE' hasta que se registre el pago.

Condición de intersección de rangos: Dos rangos de fechas $[A_{in}, A_{out}]$ y $[B_{in}, B_{out}]$ si:

$$A_{in} < B_{out} \quad y \quad A_{out} > B_{in}$$

```sql
CREATE OR REPLACE PROCEDURE sp_crear_reserva (
    p_id_cliente      IN  NUMBER,
    p_id_habitacion   IN  NUMBER,
    p_fecha_checkin   IN  DATE,
    p_fecha_checkout  IN  DATE,
    p_num_huespedes   IN  NUMBER,
    p_id_reserva      OUT NUMBER
) IS
    v_id_alojamiento  habitacion.id_alojamiento%TYPE;
    v_capacidad_max   habitacion.capacidad_max%TYPE;
    v_solapamientos   NUMBER := 0;
BEGIN
    -- 1. Validación de coherencia de fechas
    IF p_fecha_checkout <= p_fecha_checkin THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: La fecha de check-out (' ||
            TO_CHAR(p_fecha_checkout, 'YYYY-MM-DD') ||
            ') debe ser estrictamente posterior al check-in (' ||
            TO_CHAR(p_fecha_checkin, 'YYYY-MM-DD') || ').');
    END IF;
```

```sql
-- 2. Validación de existencia y capacidad de la habitación
BEGIN
    SELECT id_alojamiento, capacidad_max
      INTO v_id_alojamiento, v_capacidad_max
      FROM habitacion
     WHERE id_habitacion = p_id_habitacion;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'Error: La habitación con ID ' || p_id_habitacion || ' no existe.');
END;

IF p_num_huespedes > v_capacidad_max THEN
    RAISE_APPLICATION_ERROR(-20002, 'Error: Capacidad excedida. La habitación soporta máximo ' ||
        v_capacidad_max || ' huéspedes y se solicitaron ' || p_num_huespedes || '.');
END IF;

-- 3. Validación de disponibilidad (no solapamiento con reservas activas)
SELECT COUNT(*)
  INTO v_solapamientos
  FROM reserva_habitacion rh
  JOIN reserva r ON r.id_reserva = rh.id_reserva
 WHERE rh.id_habitacion = p_id_habitacion
   AND r.estado IN ('CONFIRMADA', 'PENDIENTE', 'COMPLETADA')
   AND rh.fecha_checkin  < p_fecha_checkout
   AND rh.fecha_checkout > p_fecha_checkin;

IF v_solapamientos > 0 THEN
    RAISE_APPLICATION_ERROR(-20003, 'Error: Habitación no disponible.
    Ya existe una reserva activa en ese rango de fechas.');
END IF;
```

```
-- 4. Inserción de cabecera (RESERVA) y detalle (RESERVA_HABITACION)
INSERT INTO reserva (
    id_cliente,
    id_alojamiento,
    fecha_creacion,
    fecha_checkin,
    fecha_checkout,
    estado,
    fecha_cancelacion
) VALUES (
    p_id_cliente,
    v_id_alojamiento,
    SYSDATE,
    p_fecha_checkin,
    p_fecha_checkout,
    'PENDIENTE',
    NULL
) RETURNING id_reserva INTO p_id_reserva;

INSERT INTO reserva_habitacion (
    id_reserva,
    id_habitacion,
    fecha_checkin,
    fecha_checkout,
    num_huespedes
) VALUES (
    p_id_reserva,
    p_id_habitacion,
    p_fecha_checkin,
    p_fecha_checkout,
    p_num_huespedes
);
```

Prueba exitosa

```
SET SERVEROUTPUT ON;
DECLARE
    v_id_nueva NUMBER;
BEGIN
    sp_crear_reserva(
        p_id_cliente     => 1,
        p_id_habitacion  => 15,
        p_fecha_checkin  => DATE '2026-11-10',
        p_fecha_checkout => DATE '2026-11-15',
        p_num_huespedes  => 2,
        p_id_reserva     => v_id_nueva
    );
    DBMS_OUTPUT.PUT_LINE('Reserva creada con exito. ID: ' || v_id_nueva);
END;
/
```

Excepción 1 (Check-out anterior o igual al Check-in)

```
DECLARE
    v_id NUMBER;
BEGIN
    sp_crear_reserva(1, 15, DATE '2026-11-15', DATE '2026-11-10', 2, v_id);
END;
/
```

```
DECLARE
*
ERROR en la línea 1:
ORA-20001: Error: La fecha de check-out (2026-11-10) debe ser estrictamente posterior al check-in (2026-11-15).
ORA-06512: en "BD_PROYECTO.SP_CREAR_RESERVA", línea 15
ORA-06512: en línea 4
```

Excepción 2 (Capacidad excedida)

```
DECLARE
    v_id NUMBER;
BEGIN
    sp_crear_reserva(1, 15, DATE '2026-11-10', DATE '2026-11-15', 10, v_id);
END;
/
```

```
DECLARE
*
ERROR en la línea 1:
ORA-20002: Error: Capacidad excedida. La habitación soporta máximo 3 huéspedes y se solicitaron 10.
ORA-06512: en "BD_PROYECTO.SP_CREAR_RESERVA", línea 33
ORA-06512: en línea 4
```

Excepción 3 (Habitación no disponible / Solape)

```
DECLARE
    v_id NUMBER;
BEGIN
    -- Se intenta reservar exactamente en las mismas fechas del caso exitoso
    sp_crear_reserva(2, 15, DATE '2026-11-12', DATE '2026-11-14', 2, v_id);
END;
/
```

```
DECLARE
*
ERROR en la línea 1:
ORA-20003: Error: Habitación no disponible.
        Ya existe una reserva activa en ese rango de fechas.
ORA-06512: en "BD_PROYECTO.SP_CREAR_RESERVA", línea 48
ORA-06512: en línea 5
```

Cursor

Se diseñó el ciclo completo con OPEN, FETCH, %NOTFOUND, %ROWCOUNT y CLOSE. Esto evidencia el control formal de recursos en memoria y permite capturar el volumen de alojamientos liquidados mediante el atributo %ROWCOUNT al momento del cierre.

Modularidad y reutilización de código: En lugar de replicar el cálculo del cruce de temporadas en SQL plano, la consulta del cursor delega el cómputo de la estadía a la función previamente construida: fn_valor_estadia(rh.id_habitacion, rh.fecha_checkin, rh.fecha_checkout). Esto garantiza consistencia contable en toda la base de datos.

Prevención de producto cartesiano en agregaciones: Una reserva puede tener múltiples líneas de habitación en RESERVA_HABITACION y simultáneamente múltiples servicios en RESERVA_SERVICIO. Realizar un JOIN directo entre ambas tablas hijas sobre la cabecera duplicaría los montos de forma artificial. El cursor resuelve esto preagrupando en dos CTEs separadas (hospedaje_res y servicios_res) a nivel de reserva antes del consolidado final.

```
CREATE OR REPLACE PROCEDURE sp_liquidar_mensual (
    p_anio            IN NUMBER,
    p_mes             IN NUMBER,
    p_id_alojamiento IN NUMBER DEFAULT NULL
) IS
    -- Rango de fechas del mes a liquidar
    v_fecha_ini DATE;
    v_fecha_fin DATE;

    -- Acumuladores globales para el reporte
    v_gran_total_reservas   NUMBER := 0;
    v_gran_total_noches     NUMBER := 0;
    v_gran_total_hospedaje  NUMBER := 0;
    v_gran_total_servicios  NUMBER := 0;
    v_gran_total_bruto      NUMBER := 0;
    v_gran_total_comision   NUMBER := 0;
    v_gran_total_neto       NUMBER := 0;

    -- Variables de cálculo por alojamiento
    v_comision NUMBER(14,2);
    v_neto     NUMBER(14,2);
```

```
CURSOR c_liquidacion (
    cp_fecha_ini      DATE,
    cp_fecha_fin      DATE,
    cp_id_alojamiento NUMBER
) IS
    WITH res_completadas AS (
        SELECT r.id_reserva,
               r.id_alojamiento
          FROM reserva r
         WHERE r.estado = 'COMPLETADA'
           AND r.fecha_checkout >= cp_fecha_ini
           AND r.fecha_checkout <  cp_fecha_fin + 1
           AND (cp_id_alojamiento IS NULL OR r.id_alojamiento = cp_id_alojamiento)
    ),
    hospedaje_res AS (
        SELECT rc.id_alojamiento,
               rc.id_reserva,
               SUM(rh.fecha_checkout - rh.fecha_checkin) AS noches,
               SUM(fn_valor_estadia(rh.id_habitacion, rh.fecha_checkin, rh.fecha_checkout)) AS valor_hospedaje
          FROM res_completadas rc
          JOIN reserva_habitacion rh ON rh.id_reserva = rc.id_reserva
         GROUP BY rc.id_alojamiento, rc.id_reserva
    ),
    servicios_res AS (
        SELECT rc.id_reserva,
               SUM(rs.cantidad * rs.precio_unitario) AS valor_servicios
          FROM res_completadas rc
          JOIN reserva_servicio rs ON rs.id_reserva = rc.id_reserva
         GROUP BY rc.id_reserva
    )
```

```
    SELECT a.id_alojamiento,
           a.nombre_comercial,
           m.nombre AS municipio,
           ta.nombre AS tipo_alojamiento,
           COUNT(DISTINCT hr.id_reserva) AS total_reservas,
           NVL(SUM(hr.noches), 0) AS total_noches,
           NVL(SUM(hr.valor_hospedaje), 0) AS total_hospedaje,
           NVL(SUM(sr.valor_servicios), 0) AS total_servicios,
           NVL(SUM(hr.valor_hospedaje), 0) + NVL(SUM(sr.valor_servicios), 0) AS total_bruto
      FROM alojamiento a
      JOIN municipio m ON m.id_municipio = a.id_municipio
      JOIN tipo_alojamiento ta ON ta.id_tipo = a.id_tipo
      JOIN hospedaje_res hr ON hr.id_alojamiento = a.id_alojamiento
      LEFT JOIN servicios_res sr ON sr.id_reserva = hr.id_reserva
     GROUP BY a.id_alojamiento, a.nombre_comercial, m.nombre, ta.nombre
     ORDER BY total_bruto DESC;

-- Registro basado en la estructura del cursor
v_reg c_liquidacion%ROWTYPE;
```

```
BEGIN
    -- 1. Validaciones de parámetros de entrada
    IF p_mes NOT BETWEEN 1 AND 12 THEN
        RAISE_APPLICATION_ERROR(-20005, 'Error: El mes debe estar comprendido entre 1 y 12.');
    END IF;

    IF p_anio < 2024 OR p_anio > 2026 THEN
        RAISE_APPLICATION_ERROR(-20006, 'Error: El año debe estar en el rango operativo del proyecto (2024-2026).');
    END IF;

    -- 2. Establecer límites temporales exactos
    v_fecha_ini := TO_DATE(p_anio || '-' || LPAD(p_mes, 2, '0') || '-01', 'YYYY-MM-DD');
    v_fecha_fin := LAST_DAY(v_fecha_ini);

    -- 3. Encabezado del reporte
    DBMS_OUTPUT.PUT_LINE('================================================================================================================================');
    DBMS_OUTPUT.PUT_LINE(' TURISMOUQ - LIQUIDACION MENSUAL DE ALOJAMIENTOS | PERIODO: ' || TO_CHAR(v_fecha_ini, 'YYYY-MM'));
    DBMS_OUTPUT.PUT_LINE('================================================================================================================================');
    DBMS_OUTPUT.PUT_LINE(
        RPAD('ID', 5) ||
        RPAD('ALOJAMIENTO', 32) ||
        RPAD('MUNICIPIO', 16) ||
        LPAD('RES', 6) ||
        LPAD('NOCHES', 8) ||
        LPAD('HOSPEDAJE', 18) ||
        LPAD('SERVICIOS', 15) ||
        LPAD('TOTAL BRUTO', 18) ||
        LPAD('COMISION(10%)', 16) ||
        LPAD('NETO ALOJ.', 18)
    );
    DBMS_OUTPUT.PUT_LINE('--------------------------------------------------------------------------------------------------------------------------------');

    -- 4. Apertura y recorrido explícito del cursor con parámetros
```

```
    OPEN c_liquidacion(v_fecha_ini, v_fecha_fin, p_id_alojamiento);
    LOOP
        FETCH c_liquidacion INTO v_reg;
        EXIT WHEN c_liquidacion%NOTFOUND;

        -- Regla de liquidación: comisión de intermediación del 10% y neto a transferir
        v_comision := ROUND(v_reg.total_bruto * 0.10, 2);
        v_neto     := v_reg.total_bruto - v_comision;

        -- Acumulación de métricas
        v_gran_total_reservas  := v_gran_total_reservas  + v_reg.total_reservas;
        v_gran_total_noches    := v_gran_total_noches    + v_reg.total_noches;
        v_gran_total_hospedaje := v_gran_total_hospedaje + v_reg.total_hospedaje;
        v_gran_total_servicios := v_gran_total_servicios + v_reg.total_servicios;
        v_gran_total_bruto     := v_gran_total_bruto     + v_reg.total_bruto;
        v_gran_total_comision  := v_gran_total_comision  + v_comision;
        v_gran_total_neto      := v_gran_total_neto      + v_neto;

        -- Salida formateada por alojamiento
        DBMS_OUTPUT.PUT_LINE(
            RPAD(TO_CHAR(v_reg.id_alojamiento), 5) ||
            RPAD(SUBSTR(v_reg.nombre_comercial, 1, 30), 32) ||
            RPAD(SUBSTR(v_reg.municipio, 1, 14), 16) ||
            LPAD(TO_CHAR(v_reg.total_reservas), 6) ||
            LPAD(TO_CHAR(v_reg.total_noches), 8) ||
            LPAD(TO_CHAR(v_reg.total_hospedaje, '999,999,999'), 18) ||
            LPAD(TO_CHAR(v_reg.total_servicios, '999,999,999'), 15) ||
            LPAD(TO_CHAR(v_reg.total_bruto,     '999,999,999'), 18) ||
            LPAD(TO_CHAR(v_comision,            '999,999,999'), 16) ||
            LPAD(TO_CHAR(v_neto,                '999,999,999'), 18)
        );
    END LOOP;
```

```
    -- Validación de registros procesados usando el atributo del cursor
    IF c_liquidacion%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No se registraron reservas completadas para liquidar en el periodo indicado.');
    ELSE
        -- 5. Pie de liquidación con totales consolidados
        DBMS_OUTPUT.PUT_LINE('-------------------------------------------------------------------------------------------------------------------------------------------------------');
        DBMS_OUTPUT.PUT_LINE(
            RPAD('TOTALES (' || c_liquidacion%ROWCOUNT || ' ALOJAMIENTOS)', 53) ||
            LPAD(TO_CHAR(v_gran_total_reservas), 6) ||
            LPAD(TO_CHAR(v_gran_total_noches), 8) ||
            LPAD(TO_CHAR(v_gran_total_hospedaje, '999,999,999'), 18) ||
            LPAD(TO_CHAR(v_gran_total_servicios, '999,999,999'), 15) ||
            LPAD(TO_CHAR(v_gran_total_bruto,     '999,999,999'), 18) ||
            LPAD(TO_CHAR(v_gran_total_comision,  '999,999,999'), 16) ||
            LPAD(TO_CHAR(v_gran_total_neto,      '999,999,999'), 18)
        );
        DBMS_OUTPUT.PUT_LINE('=======================================================================================================================================================');
    END IF;

    -- Cierre formal del cursor
    CLOSE c_liquidacion;

END sp_liquidar_mensual;
/
```

Prueba

Año (p_anio): 2025.
Mes (p_mes): 7 (Julio).
Alojamiento (p_id_alojamiento): NULL (activa la ejecución masiva de todos los establecimientos).

```
SET SERVEROUTPUT ON SIZE UNLIMITED;
SET LINESIZE 250;

BEGIN
    -- Liquidación masiva del mes de julio de 2025 (temporada alta / mitad de año)
    sp_liquidar_mensual(
        p_anio           => 2025,
        p_mes            => 7,
        p_id_alojamiento => NULL
    );
END;
/
```

ID	ALOJAMIENTO	MUNICIPIO	RES	NOCHES	HOSPEDAJE	SERVICIOS	TOTAL BRUTO	COMISION(10%)	NETO ALOJ.
4	Hotel El Mirador	Armenia	76	314	115,191,000	16,126,500	131,317,500	13,131,750	118,185,750
1	Hotel El Cafetal	Armenia	56	266	94,140,000	13,348,500	107,488,500	10,748,850	96,739,650
2	Hotel La Esperanza	Armenia	59	203	56,222,000	17,017,000	73,239,000	7,323,900	65,915,100
21	Glamping La Cascada	Salento	19	74	55,908,000	4,819,000	60,727,000	6,072,700	54,654,300
6	Hotel Buenavista	Armenia	38	149	46,745,000	10,606,000	57,351,000	5,735,100	51,615,900
23	Hotel El Recuerdo	Montenegro	38	157	48,689,000	7,485,500	56,174,500	5,617,450	50,557,050
11	Glamping El Paraiso	Armenia	17	72	49,890,000	3,800,500	53,690,500	5,369,050	48,321,450
38	Hotel La Orquidea	Quimbaya	30	117	43,035,000	7,899,000	50,934,000	5,093,400	45,840,600
13	Hotel La Cumbre	Salento	38	128	42,231,000	7,868,500	50,099,500	5,009,950	45,089,550
3	Hotel Villa Maria	Armenia	29	133	37,053,000	7,076,000	44,129,000	4,412,900	39,716,100
32	Hotel Aguas Claras	Calarca	32	123	33,026,000	7,288,500	40,314,500	4,031,450	36,283,050
47	Glamping El Retiro	Filandia	11	43	37,257,000	1,763,000	39,020,000	3,902,000	35,118,000
29	Glamping Rancho Grande	Montenegro	13	51	33,496,000	4,710,000	38,206,000	3,820,600	34,385,400
48	Hotel Casa del Arriero	La Tebaida	26	118	32,538,000	5,488,000	38,026,000	3,802,600	34,223,400
42	Glamping San Alberto	Quimbaya	13	42	35,222,000	2,673,500	37,895,500	3,789,550	34,105,950
17	Finca El Roble	Salento	17	58	29,558,000	5,050,000	34,608,000	3,460,800	31,147,200
22	Glamping Los Andes	Salento	12	45	32,056,000	1,928,000	33,984,000	3,398,400	30,585,600
19	Finca Las Palmas	Salento	22	71	28,480,000	4,226,000	32,706,000	3,270,600	29,435,400
49	Hotel El Bambu	La Tebaida	23	78	22,500,000	5,960,500	28,460,500	2,846,050	25,614,450
9	Hostal La Aurora	Armenia	43	165	21,536,000	6,479,000	28,015,000	2,801,500	25,213,500
33	Hostal La Floresta	Calarca	29	126	18,090,000	7,949,500	26,039,500	2,603,950	23,435,550
18	Finca San Isidro	Salento	11	51	21,148,000	2,962,500	24,110,500	2,411,050	21,699,450
5	Hotel La Palma	Armenia	16	64	21,708,000	2,182,500	23,890,500	2,389,050	21,501,450
34	Finca Los Cafetos	Calarca	12	47	19,206,000	4,208,000	23,414,000	2,341,400	21,072,600
46	Finca El Eden	Filandia	11	52	18,574,000	4,555,000	23,129,000	2,312,900	20,816,100
54	Finca Alma Cafetera	Circasia	12	53	20,311,000	2,769,500	23,080,500	2,308,050	20,772,450

Agrupación en paquete

La especificación PACKAGE define únicamente los contratos públicos autorizados para el resto de la aplicación , ocultando la lógica algorítmica y las consultas internas dentro del cuerpo.

```sql
CREATE OR REPLACE PACKAGE pkg_turismouq IS

    -- Constantes públicas de negocio
    c_pct_comision CONSTANT NUMBER(4,2) := 0.10;

    -- 1. Función para calcular el valor total de una estadía cruzando temporadas
    FUNCTION fn_valor_estadia (
        p_id_habitacion IN NUMBER,
        p_fecha_in      IN DATE,
        p_fecha_out     IN DATE
    ) RETURN NUMBER;

    -- 2. Procedimiento para validar y registrar una reserva con excepciones controladas
    PROCEDURE sp_crear_reserva (
        p_id_cliente      IN  NUMBER,
        p_id_habitacion   IN  NUMBER,
        p_fecha_checkin   IN  DATE,
        p_fecha_checkout  IN  DATE,
        p_num_huespedes   IN  NUMBER,
        p_id_reserva      OUT NUMBER
    );

    -- 3. Procedimiento de proceso masivo: liquidación mensual con cursor parametrizado
    PROCEDURE sp_liquidar_mensual (
        p_anio           IN NUMBER,
        p_mes            IN NUMBER,
        p_id_alojamiento IN NUMBER DEFAULT NULL
    );

END pkg_turismouq;
/
```

Prueba

```sql
DECLARE
    v_valor_calculado NUMBER;
    v_id_reserva_nueva NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('>>> 1. PRUEBA DE FUNCION EN PAQUETE: fn_valor_estadia <<<');
    v_valor_calculado := pkg_turismouq.fn_valor_estadia(
        p_id_habitacion => 10,
        p_fecha_in      => DATE '2025-12-10',
        p_fecha_out     => DATE '2025-12-15'
    );
    DBMS_OUTPUT.PUT_LINE('Valor calculado: $' || TO_CHAR(v_valor_calculado, '999,999,999'));

    DBMS_OUTPUT.PUT_LINE(CHR(10) || '>>> 2. PRUEBA DE PROCEDIMIENTO EN PAQUETE: sp_crear_reserva <<<');
    pkg_turismouq.sp_crear_reserva(
        p_id_cliente     => 2,
        p_id_habitacion  => 20,
        p_fecha_checkin  => DATE '2026-11-20',
        p_fecha_checkout => DATE '2026-11-25',
        p_num_huespedes  => 2,
        p_id_reserva     => v_id_reserva_nueva
    );
    DBMS_OUTPUT.PUT_LINE('Reserva creada con exito desde el paquete. ID: ' || v_id_reserva_nueva);

    DBMS_OUTPUT.PUT_LINE(CHR(10) || '>>> 3. PRUEBA DE LIQUIDACION MENSUAL EN PAQUETE: sp_liquidar_mensual <<<');
    pkg_turismouq.sp_liquidar_mensual(
        p_anio           => 2025,
        p_mes            => 7,
        p_id_alojamiento => 1
    );
END;
/
```

```
>>> 1. PRUEBA DE FUNCION EN PAQUETE: fn_valor_estadia <<<
Valor calculado: $   2,339,000

>>> 2. PRUEBA DE PROCEDIMIENTO EN PAQUETE: sp_crear_reserva <<<
Reserva creada con exito desde el paquete. ID: 154107

>>> 3. PRUEBA DE LIQUIDACION MENSUAL EN PAQUETE: sp_liquidar_mensual <<<
================================================================================================================================================
 TURISMOUQ - LIQUIDACION MENSUAL DE ALOJAMIENTOS | PERIODO: 2025-07
================================================================================================================================================
ID   ALOJAMIENTO                    MUNICIPIO          RES  NOCHES         HOSPEDAJE      SERVICIOS       TOTAL BRUTO   COMISION(10%)        NETO ALOJ.
------------------------------------------------------------------------------------------------------------------------------------------------
1    Hotel El Cafetal               Armenia             56     266        94,140,000     13,348,500       107,488,500      10,748,850        96,739,650
------------------------------------------------------------------------------------------------------------------------------------------------
TOTALES (1 ALOJAMIENTOS)                                56     266        94,140,000     13,348,500       107,488,500      10,748,850        96,739,650
================================================================================================================================================
```

Disparador nivel sentencia

Para registrar las modificaciones de los precios, se crea la tabla auditoria.tarifa

```sql
CREATE TABLE auditoria_tarifa (
    id_auditoria    NUMBER GENERATED BY DEFAULT AS IDENTITY,
    id_tarifa       NUMBER         NOT NULL,
    usuario         VARCHAR2(60)   NOT NULL,
    fecha_cambio    TIMESTAMP      DEFAULT SYSTIMESTAMP NOT NULL,
    operacion       VARCHAR2(10)   NOT NULL,
    valor_anterior  NUMBER(12,2)   NOT NULL,
    valor_nuevo     NUMBER(12,2)   NOT NULL,
    CONSTRAINT pk_auditoria_tarifa PRIMARY KEY (id_auditoria)
);

COMMENT ON TABLE auditoria_tarifa IS 'Registro historico de cambios de precios en las tarifas de habitaciones.';
```

En el estándar SQL de Oracle, una sentencia UPDATE masiva puede modificar miles de filas en una sola instrucción. Los pseudoregistros :OLD y :NEW no existen en un disparador de sentencia tradicional porque el motor no puede representar múltiples tuplas en una sola variable escalar.

Se implementa COMPOUND TRIGGER para resolver este conflicto arquitectónico. Permite compartir un estado común en memoria (v_cambios) entre la fase de fila (AFTER EACH ROW), donde se capturan las deltas de precio, y la fase final de sentencia (AFTER STATEMENT), donde se descarga la auditoría completa en la tabla destino.

```sql
CREATE OR REPLACE TRIGGER trg_auditoria_tarifa
FOR UPDATE OF valor_noche ON tarifa
COMPOUND TRIGGER

    -- Estructura en memoria para consolidar los cambios de la sentencia
    TYPE r_auditoria IS RECORD (
        id_tarifa      tarifa.id_tarifa%TYPE,
        valor_anterior tarifa.valor_noche%TYPE,
        valor_nuevo    tarifa.valor_noche%TYPE
    );
    TYPE t_auditoria IS TABLE OF r_auditoria INDEX BY PLS_INTEGER;
    v_cambios t_auditoria;

    -- Sección que captura los valores por cada fila modificada
    AFTER EACH ROW IS
    BEGIN
        -- Se audita únicamente si el valor efectivamente cambió
        IF :OLD.valor_noche <> :NEW.valor_noche THEN
            v_cambios(v_cambios.COUNT + 1).id_tarifa      := :OLD.id_tarifa;
            v_cambios(v_cambios.COUNT).valor_anterior     := :OLD.valor_noche;
            v_cambios(v_cambios.COUNT).valor_nuevo        := :NEW.valor_noche;
        END IF;
    END AFTER EACH ROW;

    -- Sección a nivel de sentencia: persiste todo el lote en una sola operación
    AFTER STATEMENT IS
    BEGIN
        FOR i IN 1 .. v_cambios.COUNT LOOP
            INSERT INTO auditoria_tarifa (
                id_tarifa,
                usuario,
                fecha_cambio,
                operacion,
                valor_anterior,
                valor_nuevo
            ) VALUES (
                v_cambios(i).id_tarifa,
                SYS_CONTEXT('USERENV', 'SESSION_USER'),
                SYSTIMESTAMP,
                'UPDATE',
                v_cambios(i).valor_anterior,
                v_cambios(i).valor_nuevo
            );
        END LOOP;
    END AFTER STATEMENT;

END trg_auditoria_tarifa;
/
```

Pruebas

```sql
-- 1. Consultar el valor actual de una tarifa para la prueba
SELECT id_tarifa, id_habitacion, id_temporada, valor_noche
  FROM tarifa
 WHERE id_tarifa = 1;
```

ID_TARIFA	ID_HABITACION	ID_TEMPORADA	VALOR_NOCHE
1	1	1	369000

```
-- 2. Ejecutar una actualización de precio
UPDATE tarifa
   SET valor_noche = valor_noche + 15000
 WHERE id_tarifa = 1;

COMMIT;
```

```
1 fila actualizadas.

Confirmación terminada.
```

```
-- 3. Visualizar la evidencia de auditoría registrada por el disparador
SELECT id_auditoria,
       id_tarifa,
       usuario,
       TO_CHAR(fecha_cambio, 'YYYY-MM-DD HH24:MI:SS') AS fecha,
       operacion,
       valor_anterior,
       valor_nuevo,
       (valor_nuevo - valor_anterior) AS diferencia
  FROM auditoria_tarifa
 ORDER BY id_auditoria DESC;
```

ID_AUDITORIA	ID_TARIFA	USUARIO	FECHA	OPERACION	VALOR_ANTERIOR	VALOR_NUEVO	DIFERENCIA
1	1	BD_PROYECTO	2026-10-25 23:48:11	UPDATE	369000	384000	15000

Disparador nivel fila

En Oracle, si se implementa un disparador simple BEFORE INSERT ON reserva_habitacion FOR EACH ROW que intente consultar (SELECT COUNT(*)) sobre la misma tabla reserva_habitacion para buscar solapes, el motor aborta la operación con la excepción ORA-04091 (tabla mutante) porque la tabla se encuentra en estado de modificación.

Se utiliza un disparador compuesto que opera en dos tiempos:

- En el evento a nivel de fila (BEFORE EACH ROW), captura los datos entrantes en una estructura en memoria (v_lineas).
- En el evento posterior (AFTER STATEMENT), cuando la tabla ha salido del estado de mutación, ejecuta la consulta relacional de verificación y lanza el error correspondiente si existe conflicto.

La condicion rh.id_reserva <> v_lineas(i).id_reserva previene falsos positivos al evaluar la fila que se está procesando contra sí misma. A su vez, se vincula con la cabecera RESERVA para descartar reservas con estado 'CANCELADA', las cuales liberan de inmediato el calendario operativo.

```
CREATE OR REPLACE TRIGGER trg_no_solape_habitacion
FOR INSERT OR UPDATE OF id_habitacion, fecha_checkin, fecha_checkout ON reserva_habitacion
COMPOUND TRIGGER

    -- Colección en memoria para registrar las filas procesadas en la sentencia
    TYPE r_linea IS RECORD (
        id_reserva     reserva_habitacion.id_reserva%TYPE,
        id_habitacion  reserva_habitacion.id_habitacion%TYPE,
        fecha_checkin  reserva_habitacion.fecha_checkin%TYPE,
        fecha_checkout reserva_habitacion.fecha_checkout%TYPE
    );
    TYPE t_lineas IS TABLE OF r_linea INDEX BY PLS_INTEGER;
    v_lineas t_lineas;

   BEFORE EACH ROW IS
   BEGIN
        v_lineas(v_lineas.COUNT + 1).id_reserva     := :NEW.id_reserva;
        v_lineas(v_lineas.COUNT).id_habitacion      := :NEW.id_habitacion;
        v_lineas(v_lineas.COUNT).fecha_checkin      := :NEW.fecha_checkin;
        v_lineas(v_lineas.COUNT).fecha_checkout     := :NEW.fecha_checkout;
   END BEFORE EACH ROW;
```

```
 AFTER STATEMENT IS
     v_solapes NUMBER;
     v_estado  reserva.estado%TYPE;
 BEGIN
     FOR i IN 1 .. v_lineas.COUNT LOOP
         -- Se verifica el estado de la reserva padre
         BEGIN
             SELECT estado
               INTO v_estado
               FROM reserva
              WHERE id_reserva = v_lineas(i).id_reserva;
         EXCEPTION
             WHEN NO_DATA_FOUND THEN
                 v_estado := 'PENDIENTE';
         END;

         -- Solo las reservas activas (no canceladas) comprometen disponibilidad
         IF v_estado <> 'CANCELADA' THEN
             SELECT COUNT(*)
               INTO v_solapes
               FROM reserva_habitacion rh
               JOIN reserva r ON r.id_reserva = rh.id_reserva
              WHERE rh.id_habitacion = v_lineas(i).id_habitacion
                AND rh.id_reserva   <> v_lineas(i).id_reserva
                AND r.estado        IN ('CONFIRMADA', 'PENDIENTE', 'COMPLETADA')
                AND rh.fecha_checkin  < v_lineas(i).fecha_checkout
                AND rh.fecha_checkout > v_lineas(i).fecha_checkin;
```

```
            IF v_solapes > 0 THEN
                RAISE_APPLICATION_ERROR(-20003,
                    'Error: Solapamiento detectado por el disparador. La habitación ' ||
                    v_lineas(i).id_habitacion || ' ya se encuentra ocupada entre el ' ||
                    TO_CHAR(v_lineas(i).fecha_checkin, 'YYYY-MM-DD') || ' y el ' ||
                    TO_CHAR(v_lineas(i).fecha_checkout, 'YYYY-MM-DD') || '.');
            END IF;
        END IF;
    END LOOP;
END AFTER STATEMENT;

END trg_no_solape_habitacion;
/
```

Pruebas

```
SET SERVEROUTPUT ON;

-- 1. Crear una reserva base limpia para la prueba (Habitación 30, del 1 al 5 de Diciembre de 2026)
INSERT INTO reserva (id_cliente, id_alojamiento, fecha_creacion, fecha_checkin, fecha_checkout, estado)
VALUES (1, 1, SYSDATE, DATE '2026-12-01', DATE '2026-12-05', 'CONFIRMADA');

INSERT INTO reserva_habitacion (id_reserva, id_habitacion, fecha_checkin, fecha_checkout, num_huespedes)
VALUES ((SELECT MAX(id_reserva) FROM reserva), 30, DATE '2026-12-01', DATE '2026-12-05', 2);

COMMIT;
DBMS_OUTPUT.PUT_LINE('Paso 1: Reserva base registrada exitosamente.');
```

Intento de solapamiento

```
-- 2. CASO DE PRUEBA: Intento de solapamiento directo (debe saltar el Trigger ORA-20003)
-- Se crea una nueva reserva que intenta tomar la misma Habitación 30 del 3 al 7 de Diciembre (se cruza el 3 y 4)
INSERT INTO reserva (id_cliente, id_alojamiento, fecha_creacion, fecha_checkin, fecha_checkout, estado)
VALUES (2, 1, SYSDATE, DATE '2026-12-03', DATE '2026-12-07', 'PENDIENTE');
```

```
Error que empieza en la línea: 93 del comando -
INSERT INTO reserva_habitacion (id_reserva, id_habitacion, fecha_checkin, fecha_checkout, num_huespedes)
VALUES ((SELECT MAX(id_reserva) FROM reserva), 30, DATE '2026-12-03', DATE '2026-12-07', 2)
Error en la línea de comandos : 93 Columna : 13
Informe de error -
Error SQL: ORA-20003: Error: Solapamiento detectado por el disparador. La habitación 30 ya se encuentra ocupada entre el 2026-12-03 y el 2026-12-07.
ORA-06512: en "BD_PROYECTO.TRG_NO_SOLAPE_HABITACION", línea 58
ORA-04088: error durante la ejecución del disparador 'BD_PROYECTO.TRG_NO_SOLAPE_HABITACION'
```

Caso exitoso

```
-- 3. CASO EXITOSO: Mismo cuarto en fechas posteriores no conflictivas (del 10 al 15 de Diciembre de 2026)
INSERT INTO reserva (id_cliente, id_alojamiento, fecha_creacion, fecha_checkin, fecha_checkout, estado)
VALUES (3, 1, SYSDATE, DATE '2026-12-10', DATE '2026-12-15', 'CONFIRMADA');

INSERT INTO reserva_habitacion (id_reserva, id_habitacion, fecha_checkin, fecha_checkout, num_huespedes)
VALUES ((SELECT MAX(id_reserva) FROM reserva), 30, DATE '2026-12-10', DATE '2026-12-15', 2);

COMMIT;
dbms_output.put_line('Paso 3: Reserva consecutiva registrada sin conflictos.');
```

```
1 fila insertadas.


1 fila insertadas.

Confirmación terminada.
```

Tablespace propio

Se crea el tablespace histórico de reserva junto a su tabla

```
CREATE TABLESPACE ts_historico_reservas
    DATAFILE 'ts_historico_reservas.dbf'
    SIZE 100M
    AUTOEXTEND ON NEXT 50M
    MAXSIZE 2G
    EXTENT MANAGEMENT LOCAL AUTOALLOCATE
    SEGMENT SPACE MANAGEMENT AUTO;

-- Asignar cuota al usuario del esquema TurismoUQ
ALTER USER bd_proyecto QUOTA UNLIMITED ON ts_historico_reservas;
```

```
-- ================================================================
-- 2. TABLA DE HISTÓRICO DE RESERVAS ALOJADA EN EL TABLESPACE DEDICADO
-- Almacena las reservas cerradas (COMPLETADAS o CANCELADAS) migradas
-- desde el esquema transaccional para consulta gerencial y auditoría.
-- ================================================================
CREATE TABLE reserva_historico (
    id_historico       NUMBER GENERATED BY DEFAULT AS IDENTITY,
    id_reserva         NUMBER         NOT NULL,
    id_cliente         NUMBER         NOT NULL,
    id_alojamiento     NUMBER         NOT NULL,
    fecha_creacion     DATE           NOT NULL,
    fecha_checkin      DATE           NOT NULL,
    fecha_checkout     DATE           NOT NULL,
    estado             VARCHAR2(12)   NOT NULL,
    fecha_cancelacion  DATE,
    valor_total        NUMBER(12,2)   NOT NULL,
    fecha_migracion    TIMESTAMP      DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT pk_reserva_historico PRIMARY KEY (id_historico)
)
TABLESPACE ts_historico_reservas;

COMMENT ON TABLE reserva_historico IS 'Repositorio historico de reservas finalizadas o canceladas, aislado en tablespace propio.';
```

Pruebas

```
-- 1. Verificar la existencia, estado y tamaño del tablespace en el motor
SELECT tablespace_name,
       status,
       extent_management,
       allocation_type,
       segment_space_management
  FROM user_tablespaces
 WHERE tablespace_name = 'TS_HISTORICO_RESERVAS';
```

TABLESPACE_NAME	STATUS	EXTENT_MANAGEMENT	ALLOCATION_TYPE	SEGMENT_SPACE_MANAGEMENT
TS_HISTORICO_RESERVAS	ONLINE	LOCAL	SYSTEM	AUTO

```sql
-- 2. Confirmar la ruta y configuración del Datafile asociado
SELECT tablespace_name,
       file_name,
       bytes / 1024 / 1024 AS tamano_mb,
       autoextensible,
       maxbytes / 1024 / 1024 AS maximo_mb
  FROM dba_data_files
 WHERE tablespace_name = 'TS_HISTORICO_RESERVAS';
```

TABLESPACE_NAME	FILE_NAME	TAMANO_MB	AUTOEXTENSIBLE	MAXIMO_MB
TS_HISTORICO_RESERVAS	/opt/oracle/product/26ai/dbhomeFree/dbs/ts_historico_reservas.dbf	100	YES	2048

```sql
-- 3. Confirmar que la tabla 'RESERVA_HISTORICO' reside en el nuevo tablespace
SELECT table_name,
       tablespace_name,
       status,
       logging
  FROM user_tables
 WHERE table_name = 'RESERVA_HISTORICO';
```

TABLE_NAME	TABLESPACE_NAME	STATUS	LOGGING
RESERVA_HISTORICO	TS_HISTORICO_RESERVAS	VALID	YES

```sql
-- 4. Prueba de inserción simulando la migración de una reserva histórica
INSERT INTO reserva_historico (
    id_reserva,
    id_cliente,
    id_alojamiento,
    fecha_creacion,
    fecha_checkin,
    fecha_checkout,
    estado,
    fecha_cancelacion,
    valor_total
) VALUES (
    1,
    1,
    1,
    DATE '2024-01-15',
    DATE '2024-02-01',
    DATE '2024-02-05',
    'COMPLETADA',
    NULL,
    850000.00
);

COMMIT;
```

```
1 fila insertadas.

Confirmación terminada.
```

```sql
-- 5. Visualizar el registro almacenado en el tablespace dedicado
SELECT id_historico,
       id_reserva,
       estado,
       TO_CHAR(fecha_checkin, 'YYYY-MM-DD') AS checkin,
       valor_total,
       TO_CHAR(fecha_migracion, 'YYYY-MM-DD HH24:MI:SS') AS fecha_migracion
  FROM reserva_historico;
```

ID_HISTORICO	ID_RESERVA	ESTADO	CHECKIN	VALOR_TOTAL	FECHA_MIGRACION
1	1	COMPLETADA	2024-02-01	850000	2026-10-26 00:24:24

Vista materializada

El cálculo de ocupación mensual requiere cruzar 5 entidades relacionales (MUNICIPIO, ALOJAMIENTO, HABITACION, RESERVA, RESERVA_HABITACION) y evaluar álgebra de intervalos de fechas (LEAST/GREATEST) sobre más de 25.000 reservas. La vista materializada persiste físicamente el resultado en disco (segmento propio), transformando una consulta analítica de alto consumo de I/O y CPU en un escaneo secuencial inmediato que responde en milisegundos

Se descartó el refresco incremental inmediato por commit porque introduciría una sobrecarga inaceptable en las operaciones diarias de recepción (INSERT/UPDATE en RESERVA_HABITACION). Además, el cálculo de ocupación utiliza agregaciones sobre intersecciones de rangos temporales (LEAST - GREATEST) y múltiples uniones externas, construcciones que Oracle restringe severamente para vistas materializadas de refresco rápido.

Se adopta un refresco completo bajo demanda programado mediante un Job (DBMS_SCHEDULER) para ejecutarse diariamente a las 02:00 AM, o al cierre de cada mes contable. En este horario de bajo tráfico, el motor reconstruye las 432 filas de la vista (12 municipios x 36 meses) en fracciones de segundo sin competir por bloqueos con los usuarios activos de la plataforma.

```sql
CREATE MATERIALIZED VIEW mv_ocupacion_mensual
BUILD IMMEDIATE
REFRESH COMPLETE ON DEMAND
AS
WITH meses AS (
    -- Generación de los 36 meses del periodo 2024-2026
    SELECT ADD_MONTHS(DATE '2024-01-01', LEVEL - 1) AS mes_ini,
           ADD_MONTHS(DATE '2024-01-01', LEVEL)     AS mes_fin,
           EXTRACT(YEAR FROM ADD_MONTHS(DATE '2024-01-01', LEVEL - 1))  AS anio,
           EXTRACT(MONTH FROM ADD_MONTHS(DATE '2024-01-01', LEVEL - 1)) AS mes,
           TO_CHAR(ADD_MONTHS(DATE '2024-01-01', LEVEL - 1), 'YYYY-MM') AS periodo
      FROM dual
   CONNECT BY LEVEL <= 36
),
hab_mun AS (
    -- Capacidad instalada fija por municipio (habitaciones y alojamientos)
    SELECT a.id_municipio,
           COUNT(DISTINCT a.id_alojamiento) AS total_alojamientos,
           COUNT(h.id_habitacion)           AS total_habitaciones
      FROM alojamiento a
      JOIN habitacion h ON h.id_alojamiento = a.id_alojamiento
     GROUP BY a.id_municipio
),
ocupacion AS (
    -- Intersección exacta de noches reservadas no canceladas por mes
    SELECT a.id_municipio,
           m.periodo,
           COUNT(DISTINCT r.id_reserva) AS total_reservas,
           SUM(LEAST(rh.fecha_checkout, m.mes_fin) - GREATEST(rh.fecha_checkin, m.mes_ini)) AS noches_ocupadas
      FROM meses m
      JOIN reserva_habitacion rh ON rh.fecha_checkin < m.mes_fin
                                AND rh.fecha_checkout > m.mes_ini
      JOIN reserva r ON r.id_reserva = rh.id_reserva
                    AND r.estado <> 'CANCELADA'
      JOIN habitacion h ON h.id_habitacion = rh.id_habitacion
      JOIN alojamiento a ON a.id_alojamiento = h.id_alojamiento
     GROUP BY a.id_municipio, m.periodo
)
SELECT mu.id_municipio,
       mu.nombre AS municipio,
       m.anio,
       m.mes,
       m.periodo,
       hm.total_alojamientos,
       hm.total_habitaciones,
       (m.mes_fin - m.mes_ini) AS dias_mes,
       (hm.total_habitaciones * (m.mes_fin - m.mes_ini)) AS noches_disponibles,
       NVL(o.total_reservas, 0) AS total_reservas,
       NVL(o.noches_ocupadas, 0) AS noches_ocupadas,
       ROUND(100 * NVL(o.noches_ocupadas, 0) / (hm.total_habitaciones * (m.mes_fin - m.mes_ini)), 2) AS tasa_ocupacion_pct
  FROM municipio mu
  JOIN hab_mun hm ON hm.id_municipio = mu.id_municipio
 CROSS JOIN meses m
  LEFT JOIN ocupacion o ON o.id_municipio = mu.id_municipio
 ORDER BY m.periodo, mu.nombre;

-- Comentario descriptivo en el diccionario de datos
COMMENT ON MATERIALIZED VIEW mv_ocupacion_mensual IS 'Resumen precalculado de ocupacion mensual por municipio (2024-2026).';

CREATE INDEX idx_mv_ocup_periodo ON mv_ocupacion_mensual (periodo);
CREATE INDEX idx_mv_ocup_muni    ON mv_ocupacion_mensual (id_municipio, anio, mes);
```

Pruebas

Verificar el registro de la vista en el diccionario de datos

```sql
SELECT mview_name,
       refresh_mode,
       refresh_method,
       build_mode,
       last_refresh_type,
       TO_CHAR(last_refresh_date, 'YYYY-MM-DD HH24:MI:SS') AS ultimo_refresco
  FROM user_mviews
 WHERE mview_name = 'MV_OCUPACION_MENSUAL';
```

MVIEW_NAME	REFRESH_MODE	REFRESH_METHOD	BUILD_MODE	LAST_REFRESH_TYPE	ULTIMO_REFRESCO
MV_OCUPACION_MENSUAL	DEMAND	COMPLETE	IMMEDIATE	COMPLETE	2026-10-26 00:30:03

Consulta directa sobre la vista

```sql
SELECT municipio,
       periodo,
       total_alojamientos,
       total_habitaciones,
       noches_disponibles,
       noches_ocupadas,
       tasa_ocupacion_pct || '%' AS ocupacion
  FROM mv_ocupacion_mensual
 WHERE periodo = '2025-07'
 ORDER BY tasa_ocupacion_pct DESC;
```

MUNICIPIO	PERIODO	TOTAL_ALOJAMIENTOS	TOTAL_HABITACIONES	NOCHES_DISPONIBLES	NOCHES_OCUPADAS	OCUPACION
Salento	2025-07	10	72	2232	623	27,91%
Circasia	2025-07	3	18	558	127	22,76%
Filandia	2025-07	5	36	1116	248	22,22%
Calarca	2025-07	6	57	1767	391	22,13%
Armenia	2025-07	12	210	6510	1365	20,97%
Montenegro	2025-07	8	64	1984	411	20,72%
La Tebaida	2025-07	4	37	1147	232	20,23%
Quimbaya	2025-07	6	39	1209	223	18,44%
Buenavista	2025-07	1	5	155	24	15,48%
Genova	2025-07	1	5	155	20	12,9%
Cordoba	2025-07	2	16	496	63	12,7%
Pijao	2025-07	2	11	341	11	3,23%

Demostración del PIVOT de C1 consumiendo directamente la Vista Materializada

```sql
SELECT *
  FROM (
        SELECT municipio,
               mes,
               tasa_ocupacion_pct
          FROM mv_ocupacion_mensual
         WHERE anio = 2025
       )
 PIVOT (
     MAX(tasa_ocupacion_pct)
     FOR mes IN (1 AS ene, 2 AS feb, 3 AS mar, 4 AS abr, 5 AS may, 6 AS jun,
                 7 AS jul, 8 AS ago, 9 AS sep, 10 AS oct, 11 AS nov, 12 AS dic)
 )
 ORDER BY municipio;
```

MUNICIPIO	ENE	FEB	MAR	ABR	MAY	JUN	JUL	AGO	SEP	OCT	NOV	DIC
Armenia	16,97	6,24	10,8	18,97	5,7	19,27	20,97	11,64	6,19	15,51	11,94	20,17
Buenavista	19,35	4,29	17,42	23,33	2,58	10	15,48	21,94	21,33	11,61	10,67	10,97
Calarca	17,03	7,39	8,21	17,02	5,55	17,13	22,13	12,56	4,97	12,28	9,24	16,24
Circasia	13,44	4,37	17,92	19,07	8,6	12,78	22,76	12,9	3,7	19,35	11,11	19,18
Cordoba	13,91	2,68	6,45	11,67	3,02	6,25	12,7	5,44	4,17	10,89	6,25	8,06
Filandia	16,67	6,15	7,44	15,46	5,65	18,43	22,22	11,29	8,52	11,92	6,57	19,89
Genova	7,74	0	1,29	11,33	14,19	4	12,9	5,16	0,67	2,58	0	10,97
La Tebaida	16,83	6,85	12,55	17,84	7,06	17,66	20,23	9,76	6,31	11,77	6,67	11,33
Montenegro	16,08	5,41	8,67	11,25	6,45	17,29	20,72	11,74	5,31	8,32	8,33	20,01
Pijao	5,57	0,65	5,28	2,73	1,17	7,27	3,23	5,28	8,18	0	1,21	2,35
Quimbaya	16,46	5,59	11,91	16,15	6,29	19,4	18,44	9,26	4,44	10,17	8,38	21,17
Salento	21,73	12,3	15,19	21,16	10,48	27,5	27,91	20,12	10,88	14,61	14,54	26,61